



**UNIVERSIDADE FEDERAL DE CAMPINA GRANDE (UFCG)**

**Programa:** CI EXPERT

**Aluno:** Joelson Silva de Souza

**Professor:** Henrique do Nascimento Cunha

## **Controlador de Vending Machine em SystemVerilog**

**Campina Grande – PB**

**JUIHO 2026**

## 1. Introdução

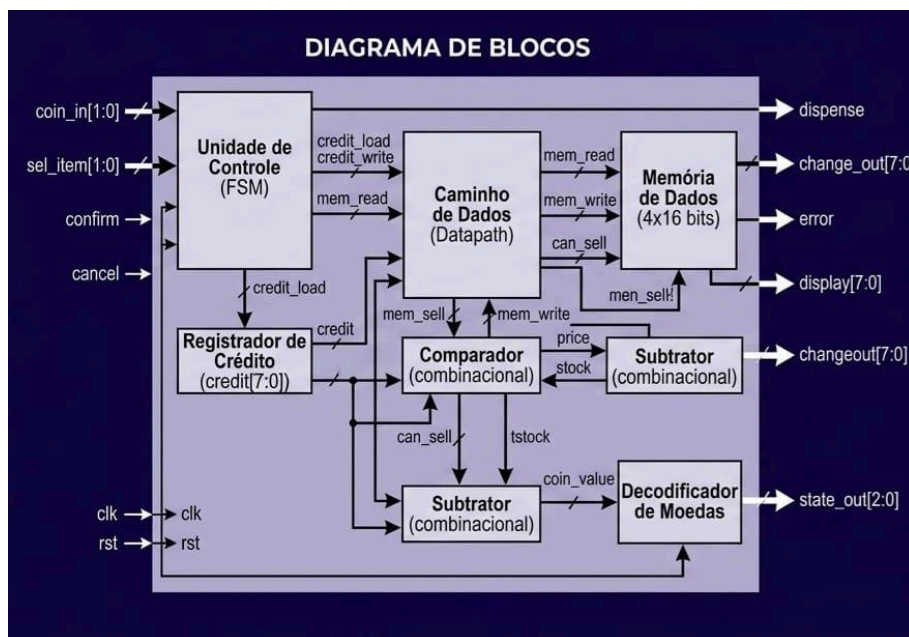
Este trabalho apresenta o projeto e implementação de uma máquina de vendas automática utilizando o SystemVerilog. O sistema foi desenvolvido de forma modular, contendo unidade de controle baseada em máquina de estados finitos (FSM), registrador de crédito, memória de preços e estoques, comparador e subtrator de troco. A verificação foi realizada por meio de testbench e análise de waveforms no DVE/Verdi, seguida de síntese lógica utilizando ferramentas Synopsys.

## 2. Link do repositório

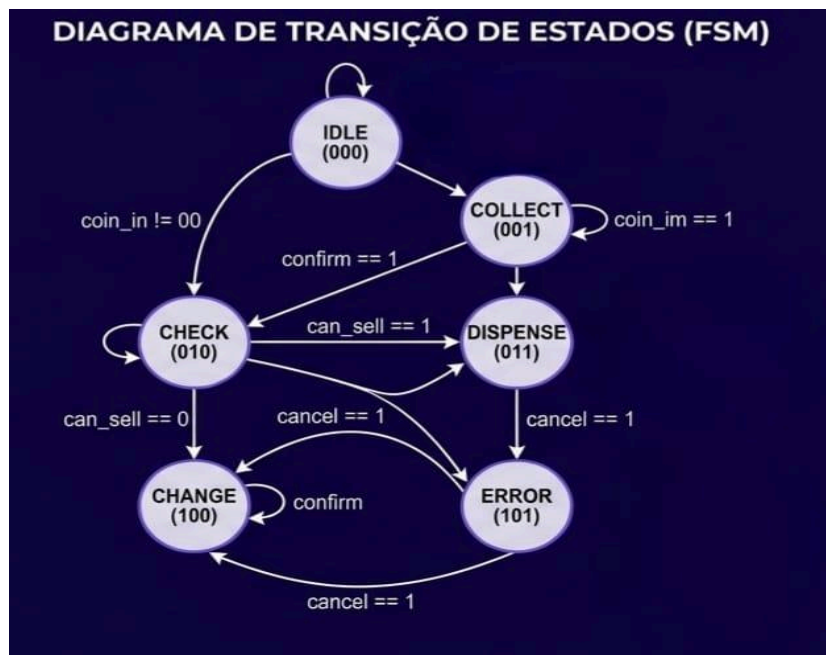
[https://github.com/seu\\_usuario/seu\\_repositorio](https://github.com/seu_usuario/seu_repositorio)

## 3. Diagrama de blocos

O sistema é composto pelos módulos control\_unit, credit\_reg, memory, comparador e subtrator, integrados no módulo vending\_top. A unidade de controle coordena a operação da máquina, enquanto os demais módulos implementam o caminho de dados.



## 4. Diagrama de estados



A FSM foi implementada com os estados IDLE, COLLECT, CHECK, DISPENSE, CHANGE e ERROR. O estado COLLECT acumula moedas, CHECK verifica disponibilidade e crédito suficiente, DISPENSE libera o produto, CHANGE calcula o troco e ERROR sinaliza falhas de operação.

## 5. Descrição dos módulos

### 5.1 Control Unit (FSM)

A **Control Unit** é responsável por controlar todo o funcionamento da máquina de vendas através de uma Máquina de Estados Finitos (FSM). Ela recebe os sinais de entrada do usuário (**coin\_in**, **confirm** e **cancel**) e o sinal **can\_sell** gerado pelo comparador. Com base nessas informações, a FSM percorre os estados **IDLE**, **COLLECT**, **CHECK**, **DISPENSE**, **CHANGE** e **ERROR**, gerando os sinais de controle **credit\_load**, **mem\_read**, **mem\_write**, **dispense**, **error** e **change\_out**. Dessa forma, a unidade de controle coordena a comunicação entre todos os módulos do sistema.

### 5.2 Credit Register

O módulo **Credit Register** implementa um registrador síncrono de 8 bits responsável por armazenar o valor total inserido pelo usuário. Cada moeda recebida em **coin\_in** é convertida para seu valor correspondente em centavos e somada ao crédito atual quando o sinal **credit\_load** é ativado pela FSM no estado

**COLLECT.** O registrador também é responsável por zerar o crédito quando ocorre um reset, um cancelamento da operação ou após a conclusão da compra no estado **CHANGE**.

### 5.3 Memory

O módulo **Memory** representa a memória da máquina de vendas. Nela são armazenados o preço e o estoque de cada produto utilizando uma palavra de 16 bits, sendo os 8 bits mais significativos destinados ao preço e os 8 bits menos significativos ao estoque. Quando o sinal **mem\_read** é ativado, a memória fornece o preço e o estoque do produto selecionado. Quando **mem\_write** é acionado após uma venda, o estoque correspondente é decrementado em uma unidade.

### 5.4 Comparador

O **Comparador** é um circuito combinacional responsável por verificar se a venda pode ser realizada. Para isso, compara o crédito acumulado pelo usuário com o preço do produto selecionado e verifica se ainda existe estoque disponível. Caso o crédito seja maior ou igual ao preço e o estoque seja maior que zero, o sinal **can\_sell** é ativado, permitindo que a FSM avance para o estado de dispensar o produto. Caso contrário, a máquina entra no estado de erro.

### 5.5 Subtrator

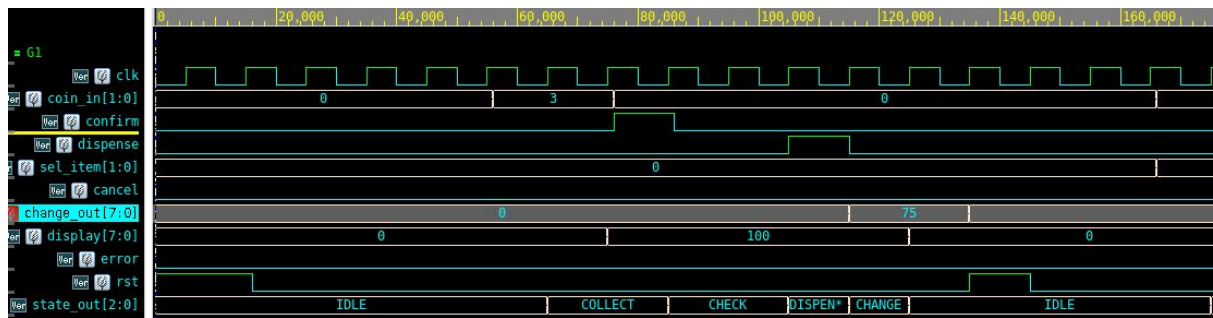
O módulo **Subtrator** implementa uma lógica combinacional responsável pelo cálculo do troco da compra. Sua função consiste em realizar a operação:

$$\text{change} = \text{credit} - \text{price}$$

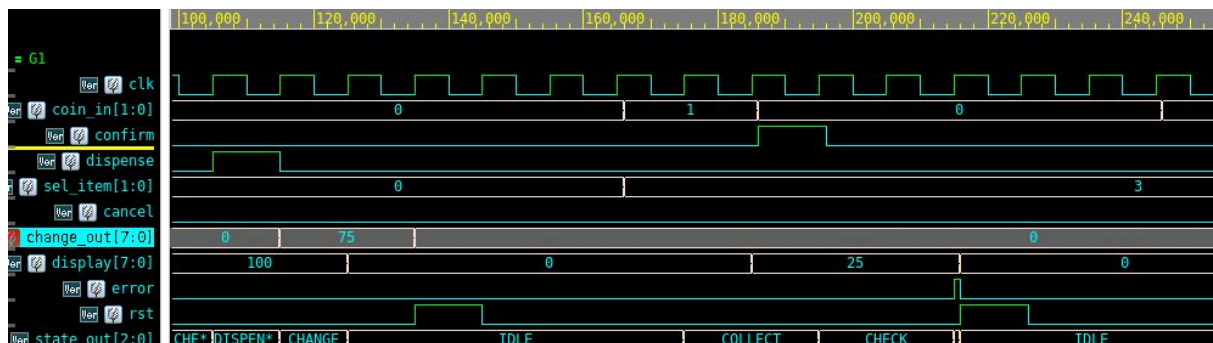
O valor calculado é disponibilizado para a unidade de controle, que o transfere para a saída **change\_out** durante o estado **CHANGE**. Dessa forma, o sistema informa ao usuário o valor do troco após a conclusão da venda.

## 6. Waveforms anotadas

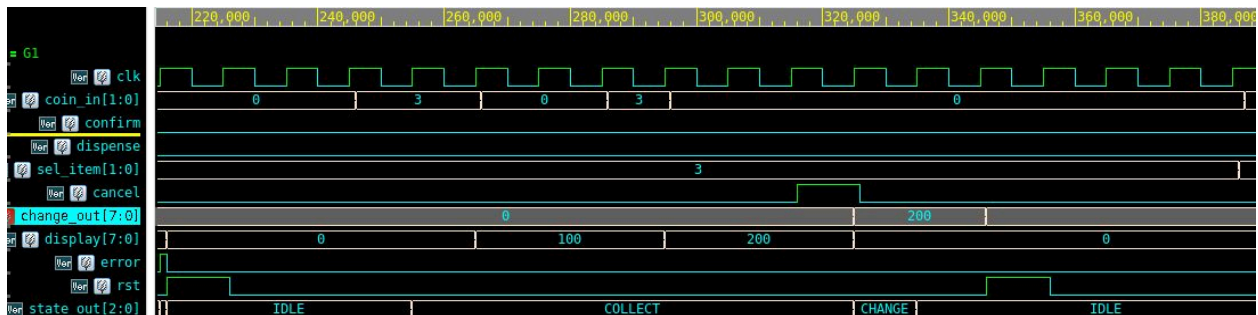
- Cenário 1 – Compra com troco: inserir moeda de R\$1,00 e selecionar café (R\$0,25).



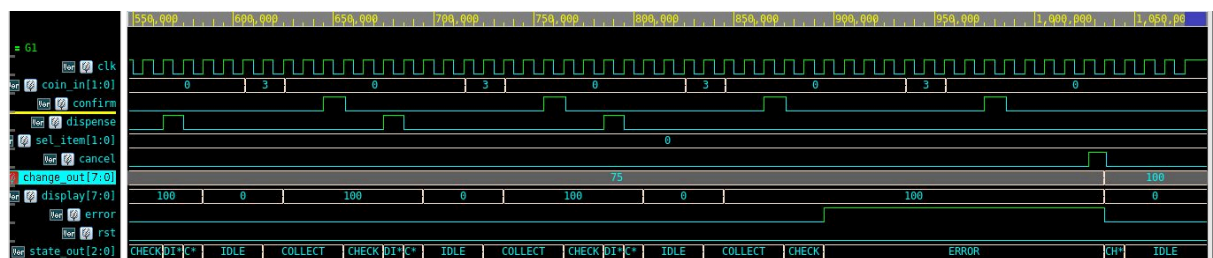
- Cenário 2 – Crédito insuficiente: inserir R\$0,25 e selecionar snack (R\$1,00).



- Cenário 3 – Cancelamento: inserir moeda e acionar **cancel**.



- Cenário 4 – Estoque zerado: realizar compras sucessivas até esgotar o estoque.



## 7. Saída do testbench

Todos os cenários definidos no testbench foram executados com sucesso, confirmando o comportamento esperado da máquina de vendas.

```
--- CENARIO 1 ---
[PASS] dispense
[PASS] troco 75

--- CENARIO 2 ---
[PASS] error ativo

--- CENARIO 3 ---
[PASS] sem dispense
[PASS] change_out=200
[PASS] credit zerado ao final

--- CENARIO 4 ---
[PASS] error estoque

SIMULACAO FINALIZADA COM SUCESSO
$finish called from file "sim/tb_vending.sv", line 192.
$finish at simulation time 1075000
V C S Simulation Report
Time: 1075000 ps
```

## 8. Resultados de síntese

| Período | Frequência | Slack(ns) | Área       | Timing   |
|---------|------------|-----------|------------|----------|
| 20ns    | 50.00      | 16.00     | 176.742352 | NET      |
| 18ns    | 55.56      | 14.00     | 176.742352 | NET      |
| 16ns    | 62.50      | 12.00     | 176.742352 | NET      |
| 14ns    | 71.43      | 10.00     | 176.742352 | NET      |
| 12ns    | 83.33      | 8.00      | 176.742352 | NET      |
| 10ns    | 100.00     | 6.00      | 176.742352 | NET      |
| 8ns     | 125.00     | 4.00      | 176.742352 | NET      |
| 6ns     | 166.67     | 2.00      | 176.742352 | NET      |
| 4ns     | 250.00     | 0.00      | 176.742352 | NET      |
| 2ns     | 500.00     | -2.00     | 176.742352 | VIOLATED |

Esse caminho atravessa o módulo **control\_unit**, passando por uma sequência de portas lógicas combinacionais (NAND, NOR e AO222) antes de chegar ao registrador **change\_r**. O maior atraso encontrado nesse caminho foi aproximadamente 3.46 ns, sendo esse o valor que limita a frequência máxima de operação do circuito.

A área não sofreu alteração porque a síntese foi realizada sobre a mesma descrição RTL e a mesma biblioteca de células. A mudança do período de clock altera apenas a restrição temporal, não a quantidade de lógica implementada

O circuito conseguiu fechar timing até um período de clock de **4 ns**, equivalente a uma frequência de: 250MHz. Ao reduzir o período para 2 ns (500 MHz), o Design Compiler não conseguiu fechar timing:

**Slack = -2.00 n e Status: VIOLATED**

O caminho crítico provavelmente está relacionado à transição envolvendo o sinal **cancel**, principalmente na lógica de controle responsável por retornar a FSM para um estado inicial ou atualizar os registradores de troco (**change\_r\_reg**).

Na FSM da máquina de vendas, a entrada **cancel** pode ser utilizada durante estados como **COLLECT**, onde o sistema precisa interromper a operação, limpar o crédito acumulado e gerar o valor de troco. Essa decisão envolve:

- análise do estado atual;
- lógica combinacional da próxima transição;
- atualização dos registradores de controle;
- atualização do registrador.

Por isso, essa transição apresenta uma cadeia lógica maior dentro da unidade de controle, resultando no maior atraso encontrado pelo sintetizador.

O caminho crítico provavelmente está relacionado à transição envolvendo o sinal **cancel**, principalmente na lógica de controle responsável por retornar a FSM para um estado inicial ou atualizar os registradores de troco (**change\_r**).

Na FSM da máquina de vendas, a entrada **cancel** pode ser utilizada durante estados como **COLLECT**, onde o sistema precisa interromper a operação, limpar o crédito acumulado e gerar o valor de troco. Essa decisão envolve:

- análise do estado atual;
- lógica combinacional da próxima transição;
- atualização dos registradores de controle;
- atualização do registrador .

Por isso, essa transição apresenta uma cadeia lógica maior dentro da unidade de controle, resultando no maior atraso encontrado pelo sintetizador.

## **9. Análise do caminho crítico**

O caminho crítico está associado à sequência registrador de crédito → comparador FSM, pois envolve operações combinacionais de comparação e decisão de estado antes da atualização dos sinais de controle. Esse caminho apresentou o menor slack entre os caminhos analisados.

## **10. Conclusão**

O projeto permitiu aplicar conceitos de FSM, caminho de dados, verificação e síntese digital utilizando SystemVerilog e ferramentas Synopsys. As principais dificuldades encontradas foram a sincronização entre o testbench e o clock, a integração entre os módulos e a depuração dos sinais de controle. Após os ajustes, todos os cenários de teste foram aprovados, demonstrando o funcionamento correto da máquina de vendas.